Do World Models improve visual, continuous control tasks?

José Andrés Ridruejo - Joaquín Mir - Javier Ríos
Final Project — Deep Reinforcement Learning

Abstract

This paper investigates the impact of World Models on visual, continuous control tasks. We aim at answering the question of whether incorporating a World Model can enhance the performance of reinforcement learning agents in complex environments (in particular, we focus on the CarRacing-v3 environment). We compare the performance of agents with and without World Models, analyzing their learning efficiency, stability, and final performance. Our results suggest that World Models can significantly improve the agent's ability to learn and perform in visual, continuous control tasks, at the cost of increased computational complexity.

1 Introduction

Over the past months, we have been working on the main advantages that including Deep Learning models in the context of Reinforcement Learning can bring. In the first project, we focused on **representation learning** [1] and how it can help to apply RL algorithms in complex environments (thus tackling the curse of dimensionality). The second project revolved around **model-free discrete control**, where we used implementations of Rainbow DQN and its variants [2], in which agents learn the *Q-functions* directly from visual observations. Finally, the third project extended this framework to **continuous control** tasks, using Actor-Critic methods, mainly PPO [3] and SAC [4], discretizing continuous action spaces and analyzing on-policy vs. off-policy trade-offs from pixel-based observations.

While these approaches demonstrated effective learning of locomotion policies in *MuJoCo* environments, they all share a fundamental limitation: **sample inefficiency**. Each gradient update relies on data collected from interactions with the environment, which becomes prohibitive in complex environments or physically constrained settings like robotics. This motivates our final project: transitioning to a **model-based** paradigm, where we can leverage a learned model of the environment to generate synthetic data and improve learning efficiency.

This final project builds on the previous ones, as we will be using similar environments and algorithms, but now we will be incorporating **World Models** [5], which are generative models that learn to predict future states of the environment. Instead of learning a policy directly from the environment, we will first learn a World Model that captures the dynamics of the environment, and then use it to train a policy. This approach has the potential to improve sample efficiency and generalization, as the agent can learn from imagined trajectories generated by the World Model.

Assuming a well-trained World Model, the agent (for example, an actor-critic algorithm) can operate solely in the compact latent space rather than the high-dimensional pixel space. This can lead to faster learning and better generalization, as the agent can focus on the underlying dynamics of the environment rather than the raw visual input. Additionally, the World Model can be used for planning and decision-making, allowing the agent to anticipate future states and make informed choices based on predicted outcomes.

We implement this using **R2-Dreamer** [6], an ICLR 2026 state-of-the-art evolution of DreamerV3 [7] that eliminates costly image reconstructions in favor of redundancy reduction in the latent space.

The core research question is clear: **can a world model outperform our model-free baselines in visual, continuous control tasks? And, if so, how does it affect sample efficiency and final performance?** We will evaluate this question in order to assess whether the added complexity of appending a World Model is justified by the performance gains it provides (which, we hypothesize, will depend on the specific environment and task at hand).

1.1 Related Work

1.1.1 Model-Free Baselines

Our model-free baselines (Rainbow DQN and PPO/SAC) are well-established algorithms in the field of reinforcement learning, known for their effectiveness in various environments. Rainbow DQN combines six improvements to the original DQN algorithm (double Q-learning, prioritized experience replay, dueling networks, multi-step learning, distributional RL, and noisy nets) for discrete action spaces, achieving strong performance in Atari games [2], but struggling with continuous control tasks.

PPO and SAC, on the other hand, are able to handle continuous actions implicitly (by means of policy gradients and maximum entropy RL) but remain sample-inefficient as each update consumes fresh rollouts without data reuse.

1.1.2 World Models

World models learn **generative dynamic models** to enable *imagination-based* policy optimization [5]. They are based (as many other ideas in the field) on the way humans learn and interact with the world:

> Humans develop a mental model of the world based on what they are able to perceive with their limited senses. The decisions and actions we make are based on this internal model … We are able to observe a scene and remember an abstract description thereof.

Early approaches to this new paradigm, such as PlaNet [8], used deterministic RNNs for latent prediction, but suffered from compounding errors and limited performance. DreamerV3 [7] (developed after 4 years by the same team) introduced some key innovations, such as the use of a **recurrent state-space model** (RSSM) with stochastic latents, and the use of **latent imagination** for policy optimization, which significantly improved performance and sample efficiency. Despite its success in DMC, Atari and Crafter environments, the decoder CNN used for image reconstruction was a major bottleneck in terms of computational cost and scalability.

R2-Dreamer [6] addresses this issue by eliminating the need for image reconstruction. Instead, it relies on **Barlow Twins-style redundancy reduction** [9], which implements an objective function that aims at making the cross-correlation matrix between two views of the latent space as close to the identity as possible. Mathematically, BT enforces:

$$\mathcal{L}_{BT} = \sum_i (1 - C_{ii})^2 + \lambda \sum_i \sum_{j \neq i} C_{ij}^2 \tag{1}$$

where C is the cross-correlation matrix between two views of the latent space, and λ is a hyperparameter that controls the trade-off between invariance and redundancy reduction. This produces compact, decorrelated latents without the need for pixel-level supervision, thus enabling a **1.6× training speedup** over DreamerV3 while maintaining or improving performance across a wide range of tasks.

More recently, other approaches such as Next Embedding-Dreamer [10] and ResWM [11] have further improved the performance of world models by introducing new architectures and training techniques, such as contrastive learning and residual connections. In spite of this, R2-Dreamer remains the candidate par excellence for our project, as it strikes a good balance between performance and computational cost.

2 Method

In order to evaluate the impact of World Models on visual, continuous control tasks, we use the code developed by the authors of R2-Dreamer [6], which is publicly available on GitHub[1]. This implementation is highly efficient, achieving a 8× speedup over the original DreamerV3 implementation, and is compatible with a wide range of environments

2.1 R2-Dreamer Architecture

The codebase is built on top of a PyTorch reproduction of DreamerV3. As such, it keeps the overall world-model/actor-critic structure intact, while replacing the image reconstruction objective with the redundancy reduction objective we described earlier. From a high-level perspective, the architecture consists of the following components:

- **Latent Dynamics Model**: This is the core of the world model, which learns to predict future latent states based on the current state and action. It consists of a recurrent neural network (RNN) that captures the temporal dependencies in the environment, and a stochastic latent variable that captures the uncertainty in the predictions.
- **World Model Learning**: This component is responsible for training the latent dynamics model using the Barlow Twins-style redundancy reduction objective. It takes pairs of latent states and computes the cross-correlation matrix, which is then used to compute the loss function that encourages informative and decorrelated features.

  Its objective function, derived from the original DreamerV3 objective, is modified to:

  $$\mathcal{L} = \beta_{BT}\mathcal{L}_{BT} + \sum_t \left(\mathcal{L}_{pred}(t) + \beta_{dyn}\mathcal{L}_{dyn}(t) + \beta_{rep}\mathcal{L}_{rep}(t)\right) \tag{2}$$

  This objective removes the reconstruction term and replaces it with the Barlow Twins loss, while keeping the prediction, dynamics, and representation losses to ensure that the latent space captures the necessary information for control.

  The BT component is computed as follows:

  $$\mathcal{L}_{BT} = \sum_i (1 - C_{ii})^2 + \alpha \sum_{i \neq j} C_{ij}^2 \tag{3}$$

  where C_{ij} is the cross-correlation between features i and j, and α is a hyperparameter that controls the strength of the redundancy penalty. The first term, therefore, can be interpreted as an invariance term that encourages the diagonal elements of the cross-correlation matrix to be close to 1, while the second term is a redundancy reduction term that encourages the off-diagonal elements to be close to 0.
- **Actor-Critic Learning**: In order to respect the antediluvian rule of **ceteris paribus**, the authors keep the actor-critic architecture and learning algorithm unchanged from DreamerV3. The only difference is that the actor and critic networks now operate in the latent space learned by the world model, rather than the pixel space. Specifically, the critic is optimized on both imagined rollouts (using the latent dynamics model) and replay trajectories (using the actual environment), while the actor is optimized only on imagined trajectories.

[1] r2dreamer

To better understand the architecture, we include a figure from the original paper (Figure 1), which illustrates the representation learning process in R2-Dreamer. In the diagram, o_t denotes the current observation, a_{t-1} the previous action, and s_t the latent state learned by the world model. The latent state is decomposed into a deterministic component h_t and a stochastic component z_t, which together capture the dynamics of the environment. The encoder maps the observation into a representation space, while the projector is used to compute the Barlow Twins-style redundancy-reduction loss L_{BT}, encouraging informative and decorrelated latent features.

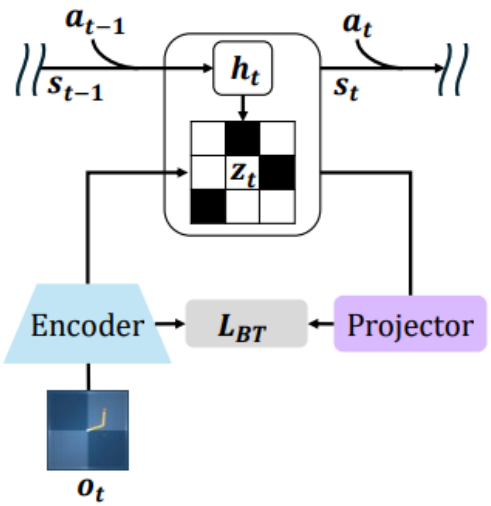


Figure 1: Representation learning in R2-Dreamer (figure obtained from [6]). Instead of a decoder, a Projector component is used to align the latent state s_t with the embedding of the observation o_t through a Barlow Twins-style objective.

3 Experimental Setup

Our experiments go along two different axes: first, we evaluate the performance of R2-Dreamer on environments that we have previously worked with (Hopper-v3) in order to compare it with our baseline, and extend to similar but more complex ones (Walker2d-v3). Second, we evaluate the performance of R2-Dreamer on the CarRacing-v3 environment, which is a visual, continuous control task that we have not previously worked with, and where we expect the benefits of the world model to be more pronounced.

In Table 1, we summarize the main hyperparameters used in our experiments, which are based on the original R2-Dreamer implementation and adapted to our computational resources:

Table 1: Essential hyperparameters for R2-Dreamer across different environments

Parámetro	**walker_walk**	**hopper_hop**	**car_racing2**
`action_repeat`	2	2	4
`time_limit`	1000	1000	10000
`steps`	1010000	1010000	inf
`batch_size`	32	16	32
`batch_length`	32	64	32
`imag_horizon`	8	15	8
`env_num`	4	4	4
`eval_episode_num`	10	10	10

Apart from these, we use the same **learning rate** (4×10^{-5}), **gradient clip** (1000.0) and **policy entropy coefficient** (3×10^{-4}) across all environments, as they are the default values in the original implementation and we did not find it necessary to tune them for our experiments. Let's briefly explain the reasoning behind some of these hyperparameters:

- `action_repeat`: This parameter controls how many times the same action is repeated in the environment before taking a new action. In visual environments like CarRacing, a higher action repeat can help to reduce the frequency of decision-making and allow the agent to focus on more meaningful state transitions, while in simpler environments like Hopper and Walker, a lower action repeat can help to capture finer-grained dynamics (thus the difference between 2 and 4).

- `time_limit`: This parameter sets the maximum number of steps per episode. In CarRacing, we set a higher time limit to allow the agent to explore the environment more thoroughly and learn from longer trajectories, while in Hopper and Walker, a lower time limit is sufficient to capture the relevant dynamics of the task (since these dynamics are much simpler and can be learned in shorter episodes).

- `steps`: This parameter constrains the total number of training steps. In CarRacing, we set it to infinity to allow the agent to train until convergence, while in Hopper and Walker, we set it to a fixed number (1 million) to ensure that the training process is manageable and does not take too long (and also to allow for a fair comparison with our baselines, which were trained for a similar number of steps).

- `imag_horizon`: This parameter tunes the length of the imagined trajectories used for policy optimization. Since Hopper is a simpler environment with more deterministic dynamics, we can afford to use longer imagined trajectories (15 steps) without suffering from compounding errors, while in Walker and CarRacing, which are more complex and have more stochastic dynamics, we use shorter imagined trajectories (8 steps) to mitigate the impact of model inaccuracies.

- `steps`: We let the training run for unlimited time in the case of CarRacing, as we want to allow the agent to train until convergence, while in Hopper and Walker, we set a fixed number of steps (1 million) to ensure that the training process is manageable and does not take too long.

4 Results

Due to time constraints, we were only able to run the experiments for a few thousand epochs, which is not enough to reach convergence in most of the environments but the simplest ones (like Hopper and Walker). However, we can still analyze the training curves and the behavior of the agents in order to draw some preliminary conclusions about the performance of R2-Dreamer in these environments.

4.1 Walker Walk

Walker Walk is a simplified version of the Walker2d environment, where the agent's objective is to walk forward as fast as possible. It is a relatively challenging environment since it uses pixel observations and has a high-dimensional action space, but it is still simpler than the original Walker2d environment, which makes it a good candidate for testing the performance of R2-Dreamer in a visual task.

In Figure 2, we can see the training curve for R2-Dreamer on the Walker Walk environment.

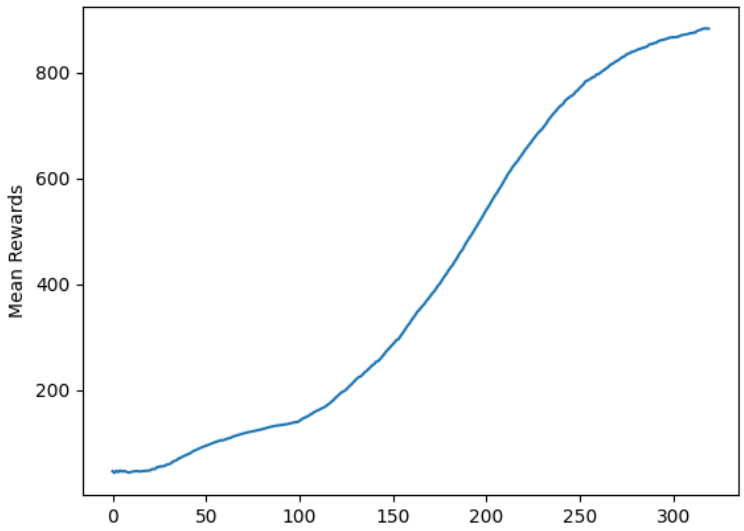


Figure 2: Training curve for R2-Dreamer on the Walker Walk environment (X-Axis in the thousands). The graph shows that the training is stable and hasn't yet converged after only 300,000 steps, which is a good sign of the sample efficiency of the algorithm.

As per the Control Suite documentation, the reference reward for this environment is around 968. Thus, even though the training curve hasn't converged, we can hypothesize that it would asymptotically reach a reward close to that value in a few more hundred thousand steps. Something else to note about this training is that the 300,000 steps took only around 20 minutes to run (this is, it ran at an average of around 250 steps per second).

Even though we neither have been able to run the baselines under this environment (once again, due to time constraints) nor have we found any reference in the literature about the performance of model-free algorithms in this specific environment, we can use the insights obtained during the previous projects to hypothesize the performance of the baselines. Specifically, using the Hopper environment, PPO took around 9 hours of training to converge, while SAC was a bit faster at around 6 hours.

Given that Walker is a more complex environment than Hopper, we can expect the baselines to take even longer to converge, which would further highlight the sample efficiency of R2-Dreamer in this environment. Even if Dreamer took 40 more minutes (this is, took double the time to reach convergence from the current point than it took to reach current performance), it would still be significantly faster than the baselines, which shows the potential of world models to improve sample efficiency.

While we have some preliminary insights about the training curve, we can also analyze the behavior of the agent in order to draw some conclusions about the performance of R2-Dreamer in this environment. In Figure 3, we can see a set of 4 frames from a video of the best performance checkpoint [2]

[2] Since LaTeX doesn't allow us to include videos, we had to extract a few frames from the video and include them as images. The video itself is available in the GitHub repository, under the `/runs/walker_walk` folder.

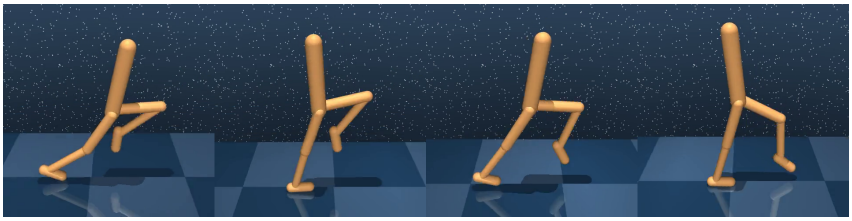


Figure 3: Set of 4 frames from the Walker Walk environment, extracted from a video of the best performance checkpoint. The agent seems to achieve its objective of walking forward and reaching the right side of the screen, though not in a purely orthodox way, as it seems to be forgetting about its right leg and dragging it behind, which is a common failure mode in this environment.

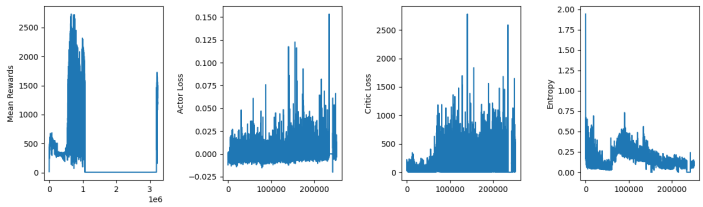
These frames show the agent walking forward and reaching the right side of the screen [3]. This is a good sign that the agent is learning to achieve its objective. That comes with a caveat, though: the agent seems to be tricking the system by only using its left leg to walk forward, while dragging its right leg behind. This is not only interesting from a behavioral perspective, but also gives us a glimpse into the inner workings of reward functions and how they can be exploited by the agent in order to achieve high rewards without necessarily learning the intended behavior.

In this case, we can interpret the result as a signal that we haven't been able to actually guide the agent towards learning what we would understand as "walking", but rather towards learning a specific behavior that happens to achieve high rewards (as it allows the agent to move forward and reach the right side of the screen, having to take less variables into account and thus being easier to learn). This is a common issue in reinforcement learning, and it highlights the importance of designing reward functions that are aligned with the intended behavior, as well as the potential benefits of using world models to learn more robust and generalizable representations of the environment.

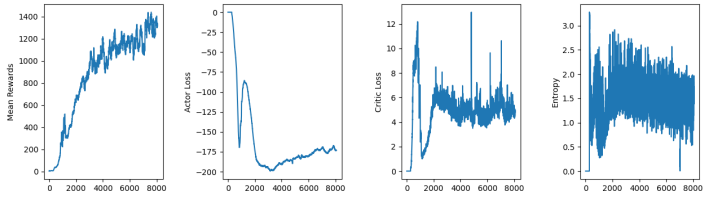
4.2 Hopper Hop

This is one of the main environments we have worked with in the previous projects, and it is a good candidate for testing the performance of R2-Dreamer. The agent's objective in this environment is to hop forward as fast as possible (while only using one leg), which is a relatively simple task compared to Walker, but still requires the agent to learn a complex coordination of its joints in order to achieve high rewards. In order to do a little recap on the results we obtained in the previous projects, we include the training curve for PPO and SAC on the Hopper environment in Figure 4.

[3]The frames belong, each, to a different quarter of the video, being shown in temporal order. This will be the case, as well, for the rest of environments



(a) PPO



(b) SAC

Figure 4: Training curves for PPO and SAC on the Hopper environment (X-Axis in thousands).

Something to remember about these two cases is the time they took to run: PPO's execution was maintained for more than a week, even though the training curve seemed to have converged after 3 days (peaking at a reward of 2500). SAC, on the other hand, was only executed for 16 hours (time period after which we cannot guarantee that the training process had converged, as the reward was still increasing and was cut at around 1500). As such, we concluded that SAC was far more sample-efficient than PPO in this environment. Now, we will include the training curve for R2-Dreamer on the same environment in Figure 5.

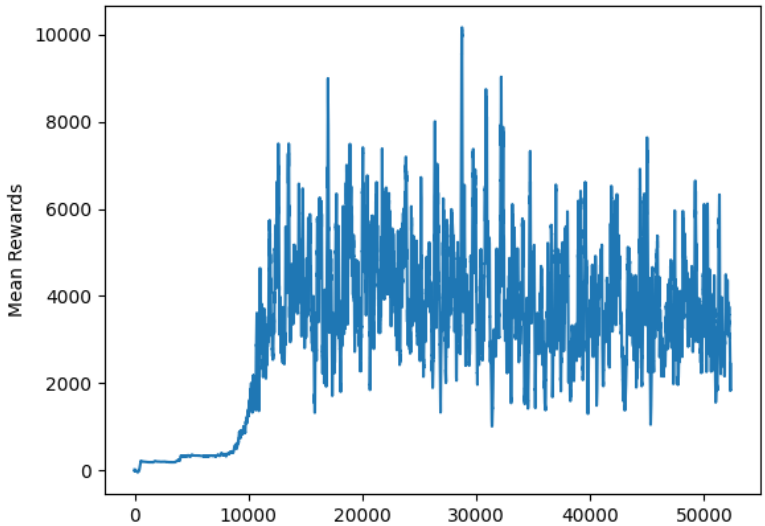


Figure 5: Training curve for R2-Dreamer on the Hopper environment (X-Axis in the thousands). The training is incredibly fast and stable, reaching peak performance in just 3,000,000 steps.

The complete training process for R2-Dreamer yielded better results across the board. Remarkably, the agent reached the end of the training goals in less than half an hour, achieving optimal performance in just 10,000 episodes and 3,000,000 steps. Thus, the training time for R2-Dreamer is significantly lower than the one for PPO and SAC, achieving a massive speed-up. In order to ground these results, we can also include a set of 4 frames from a video of the best performance checkpoint in the Hopper environment (Figure 6).

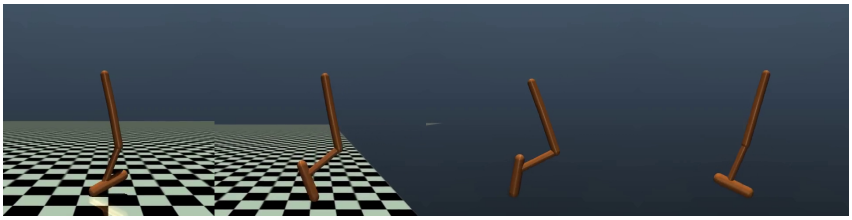


Figure 6: Set of 4 frames from the Hopper environment, extracted from a video of the best performance checkpoint.

The performance of R2-Dreamer in this environment is the best compared to the baselines (we include both best_videos in the GitHub repository for comparison, under the `/runs/hopper_v5/eval` folder, with the names `Hopper_vid_R2.mp4` and `sac_hopper.mp4`). Not only did it completely outperform our previous baselines, but it even surpassed algorithms like Rainbow DQN, which inherently count on the advantage that discretizing actions in this environment does not penalize or hinder learning.

There are several things we can definitively conclude about this case. The world model's sample efficiency completely overshadows model-free approaches here, demonstrating that the benefits of learning the environment's dynamics far outweigh the computational overhead of imagining it. Consequently, even in simpler environments where dynamics are relatively easy to learn and the reward function is dense, model-based approaches like R2-Dreamer can easily dominate and prove to be vastly superior to model-free algorithms.

4.3 Car Racing

In the previous sections, we have focused on what we could understand as "simple" environments, where the agent has to focus on things like balance or coordination in order to achieve its objective. In this section, we will analyze the performance of R2-Dreamer in a more complex environment, where the agent has to learn to drive a car around a track (in a 2D setting, where the view is from above) and reach the finish line. This is both a visually and temporally harder task, since the agent must follow the track geometry and handle longer-term dependencies in order to reach the finish line.

What's more, the tracks are procedurally generated, this is, they're different in each episode, which adds an extra layer of complexity to the task, as the agent has to learn a more generalizable policy that can adapt to different track geometries and not just memorize a specific one (as, it could be argued, is the case in previous environments, where no elements changed between iterations but the policy itself). Added to this, the agent has to smoothly coordinate the actions (steering, gas and brake), especially in settings such as curves.

Last, the reward function in this environment is quite sparse, as the agent simply gets a small penalty each frame, a big reward for crossing the finish line, and a big penalty for going off track. This makes the learning process more difficult, as the agent has to explore the environment and learn from sparse feedback in order to achieve high rewards.

In Figure 7, we can see the training curve for R2-Dreamer on the CarRacing environment. Contrary to the previous case, the training curve is a little bit more noisy, which is expected given the complexity of the environment and the fact that it is a visual task. However, we can still see a clear upward trend in the rewards, seeming to converge after just a few million steps, which is a remarkable result given the complexity of the task.

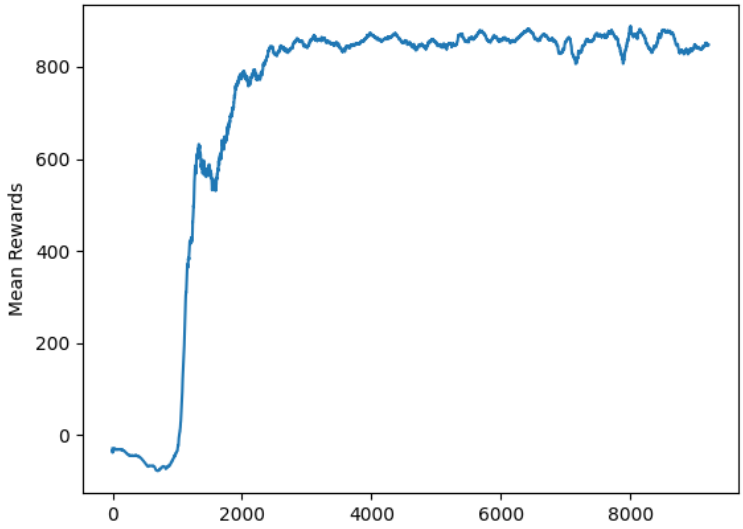


Figure 7: Training curve for R2-Dreamer on the CarRacing environment (X-Axis in the thousands). A clear uptrend can be observed, with convergence after just a few million steps.

Analyzing this previous training curve, we can hypothesize that the agent is learning to drive around the track and reach the finish line. It takes around 2.5 million steps to reach a stable ground in the training curve. In terms of computational expenditure, these 2.5 million steps took a bit less than 6 hours to run. As we did in the case of Walker (4.1), we can use the insights obtained during the previous projects to hypothesize the performance of the baselines in this environment. In order to do so, we must first set some statements:

1. The CarRacing environment is significantly more complex than the previous environments we have worked with, both in terms of visual complexity and temporal dependencies.
2. Model-free algorithms like PPO and SAC are known to be sample-inefficient, especially in visual environments with sparse rewards, as they rely on learning directly from interactions with the environment without leveraging any model of the dynamics.

Taking this into account, instead of hypothesizing the exact training time for the baselines, we can set a conservative **lower bound** for their training time. Since we know that PPO and SAC took, respectively, around 9 and 6 hours to converge in the Hopper environment, and our model-based approach took around 6 hours to reach a stable ground in the training curve in CarRacing, we can hypothesize that, granted that the baselines would also reach convergence in CarRacing (which is a big assumption, given the complexity of the environment), they would take far more than 6 hours to do so, which would further highlight the sample efficiency of R2-Dreamer in this environment.

In the following figure, as we did for the previous environments, we can see a set of 4 frames from a video of the best performance checkpoint in the CarRacing environment.

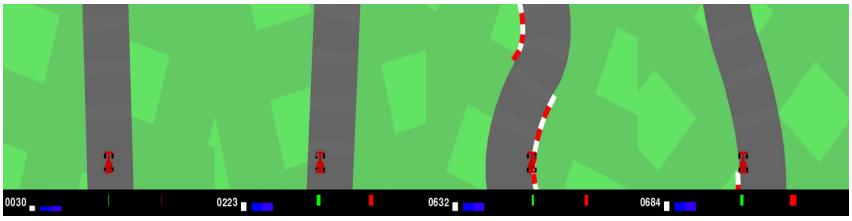


Figure 8: Set of 4 frames from the CarRacing environment, extracted from a video of the best performance checkpoint. The agent drives around the track and reaches the finish line, which is a remarkable result given the complexity of the task and the fact that it is a visual environment.

These frames are far less informative than the ones from the previous environments, as they don't show any specific behavior that we can analyze in detail (thus, we recommend any reader to further analyze the videos in the GitHub repository), but the strip shows the car staying on track, handling curves without cutting into the grass, and eventually reaching the finish line.

5 Limitations and Discussion

As we have previously mentioned, the main limitation of our experiments is the fact that we were not able to run them for a long enough time to reach convergence in most of the environments, which limits our ability to draw definitive conclusions about the performance of R2-Dreamer in these environments.

Additionally, some additional experiments would have been quite rewarding (pun intended), such as running the baselines in the same environments in order to have a direct comparison between model-based and model-free approaches, or performing an ablation study to analyze the impact of different components of the R2-Dreamer architecture (such as the Barlow Twins loss or the imagined rollouts) on the performance of the agent.

Despite this, we are optimistic about the results we have obtained, as they show a clear improvement in sample efficiency and performance compared to what we would expect from model-free approaches, especially in the more complex CarRacing environment. This suggests that incorporating a World Model can indeed enhance the performance of reinforcement learning agents in visual, continuous control tasks, at the cost of greater computational demands (in terms of resources, albeit not necessarily in terms of time, as the training times we observed were quite reasonable given the complexity of the tasks).

This suggests a simple heuristic: if interaction is cheap and the task is low-dimensional or has dense rewards, model-free methods like SAC remain an attractive default. When interaction is expensive, observations are high-dimensional, or generalization across many task configurations is required, investing in a world model becomes more appealing

A last topic to mention is the relationship between the difference in performance between R2-Dreamer and the baselines across the Hopper environment. The fact that the behavior is much more erratic in Dreamer's case (5) may be connected to how the world model optimizes in the latent space, which may lead to more unstable learning dynamics. Small model inaccuracies can lead to compounding errors in the imagined trajectories, which can cause the policy optimization to be more unstable.

6 Conclusion

In this project, we have explored the use of World Models in visual, continuous control tasks, using R2-Dreamer as our main algorithm. We have evaluated its performance in three different environments (Walker Walk, Hopper Hop and CarRacing), both in absolute terms (analyzing the training curves and the behavior of the agents) and in relative terms (in the case of Hopper, comparing it with the baselines we obtained in the previous projects).

Overall, our results suggest that World Models are quite effective in improving sample efficiency and performance in visual, continuous control tasks. Despite this, they cannot be misinterpreted as a one-for-all solution, as we observe that in simpler environments like Hopper, the benefits of using a World Model may not be as pronounced, and model-free approaches may still be competitive or even superior.

However, in more complex environments like CarRacing, the benefits of using a World Model are quite clear, as it allows the agent to learn a more generalizable policy that can adapt to different track geometries and handle longer-term dependencies, which is something that model-free approaches may struggle with.

References

[1] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119, pages 1597–1607. PMLR, 2020. doi: 10.48550/arXiv.2002.05709. URL `https://proceedings.mlr.press/v119/chen20j.html`.

[2] Matteo Hessel, Joseph Modayil, Hado Van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad G. Azar, and David Silver. Rainbow: Combining improvements in deep reinforcement learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1), 2018.

[3] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[4] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *arXiv preprint arXiv:1801.01290*, 2018.

[5] David Ha and Jürgen Schmidhuber. World models. *arXiv:1803.10122*, 2018.

[6] Takahiro Morihira et al. R2-dreamer: Redundancy-reduced world models without decoders or augmentation. In *ICLR 2026*, 2026.

[7] Danijar Hafner et al. Mastering diverse control tasks through world models. *Nature*, 2023. doi: 10.1038/s41586-025-08744-2.

[8] Danijar Hafner et al. Learning latent dynamics for planning from pixels. *International Conference on Machine Learning*, 2019.

[9] Jure Zbontar, Li Jing, Ishan Misra, Yann LeCun, and Stéphane Deny. Barlow twins: Self-supervised learning via redundancy reduction. *Proceedings of the 38th International Conference on Machine Learning*, 2021.

[10] George Bredis, Nikita Balagansky, and Ruslan Rakhimov. Ne-dreamer: Next embedding prediction makes world models stronger, 2026. URL `https://arxiv.org/abs/2603.02765`.

[11] Jseen Zhang, Gabriel Adineera, Jinzhou Tan, and Jinoh Kim. Reswm: Residual-action world model for visual rl, 2026. URL `https://arxiv.org/pdf/2603.11110`.